# ICS Exercise — Week 7: Solutions

April 16, 2026

## Problem 1: Signal Handler Concurrency

### 1.1 Visibility table

| Output line | Visibility |
|---|---|
| L09 `reaped\n` | **T** |
| L17 `u1\n` | **T** |
| L39 `loop done\n` | **T** |
| L43 `usr1_n=...\n` | **M** |

- L09 prints once per successful `waitpid` return, and every one of the 3 children exits exactly once and is reaped exactly once → printed at least once (in fact exactly 3 times).
- L17 prints once per `handler_usr1` invocation, and the very first SIGUSR1 from a child is always delivered (SIGUSR1 is not blocked), so the line prints at least once.
- L39 sits on the main path after a finite `for` loop with no exits and no signals masked that would block reaching it; always executes.
- L43 is reached only if the `while (reaped_n < 3) Pause();` loop terminates. Because of the race in 1.3, there is a legal interleaving in which `Pause()` blocks forever — under that interleaving L43 is never printed. Hence **M**.

### 1.2 Possible values

a. `reaped_n` after the loop is **exactly 3**. Each of the three children exits exactly once, so `waitpid(-1, NULL, WNOHANG)` returns a positive pid exactly three times across all `handler_chld` invocations, and each return increments `reaped_n` once. The loop's exit condition (`reaped_n >= 3`) cannot overshoot because there are no more children to reap.

b. `usr1_n` printed by line 43 is in **{1, 2, 3}**. Three children each send one SIGUSR1, but pending signals are not queued — the kernel maintains at most one pending bit per signal type. If all three sends happen while SIGUSR1 is currently being handled (or is already pending), the second and third coalesce into the existing pending state, so the handler runs only once or twice in total. Specifically:

- If sends are well-spaced (each handler returns before the next signal arrives): `usr1_n = 3`.
- If the second arrives while the first handler runs and the third arrives after the first handler returns: `usr1_n = 3` or `2` depending on timing.
- If both the second and third arrive while the first handler runs: `usr1_n = 2`.
- If all three sends arrive while SIGUSR1 is already pending (e.g., the parent has not yet been scheduled to deliver any of them): `usr1_n = 1`.

### 1.3 Race condition

**Hanging interleaving.** Suppose two children have already exited and been reaped, so `reaped_n == 2`. The third child has not yet exited.

1. Main evaluates `reaped_n < 3` → true.

2. Just before main calls `Pause()`, the third child exits, the kernel delivers SIGCHLD, and `handler_chld` runs to completion, setting `reaped_n = 3`.
3. Main now calls `Pause()`. There are no more children, no more signals will ever arrive. `Pause()` blocks forever and L43 is never printed.

**Fix using `sigsuspend`.** Keep SIGCHLD blocked across the loop and use `sigsuspend(&prev)` to atomically unblock SIGCHLD and wait:

```
/* line 30 unchanged: Sigprocmask(SIG_BLOCK, &mask_chld, &prev);  */
/* line 40 removed (do NOT unblock before the wait loop)          */
/* line 42 replaced:                                              */
while (reaped_n < 3)
    sigsuspend(&prev);                /* unblock + pause atomically */
Sigprocmask(SIG_SETMASK, &prev, NULL); /* restore original mask    */
```

`sigsuspend(&prev)` swaps the process mask to `prev` (SIGCHLD now allowed), pauses, and on signal handler return atomically restores the mask back to "SIGCHLD blocked." The check-then-pause window is closed — between iterations of the `while`, SIGCHLD is blocked, so a delivery cannot slip in and be lost.

### 1.4 Async-signal safety

The proposal violates **G1 — call only async-signal-safe functions from a handler**. `printf` is not on the POSIX async-signal-safe list because it acquires an internal `FILE*` lock and writes into a user-space buffer.

Concrete consequence: if `main` is in the middle of `printf("usr1_n=%d\n", usr1_n)` (line 43) holding the stdout lock, and SIGCHLD is delivered, the handler also calls `printf`. The handler then either (a) deadlocks waiting for a lock its own thread already holds, or (b) on a non-locking libc, interleaves bytes into the stdout buffer and produces garbled output. The lecture's `sio_puts` writes directly via the `write` syscall — itself async-signal-safe — and bypasses stdio entirely, so neither problem can occur.

(In `main`, `printf` is fine because `main` is the only writer to stdout *outside* of handlers, and at the point of L43 the wait loop has already terminated.)

---

## Problem 2: Safe Recovery with `sigsetjmp` / `siglongjmp`

### 2.1 Output trace

```
sigsetjmp returned 0
starting work
about to raise SIGINT
handler
sigsetjmp returned 1
restart #1
about to raise SIGINT
handler
sigsetjmp returned 2
restart #2
about to raise SIGINT
handler
sigsetjmp returned 3
giving up after 3 restarts
```

### 2.2 Why line 31 is unreachable

`Kill(getpid(), SIGINT)` on line 30 delivers SIGINT to the process synchronously. The kernel transfers control to `handler`, which prints `handler` and then calls `siglongjmp(env, restarts)`. `siglongjmp` does not return — it restores the saved CPU state to the values captured by `sigsetjmp` on line 17, so execution resumes with `sigsetjmp` returning `restarts` (a nonzero value). Control never returns from the handler to the kernel and never resumes at line 31.

### 2.3 `setjmp` vs. `sigsetjmp`

When the kernel invokes `handler` on the first iteration, it adds SIGINT to the process's signal mask so that the same signal cannot recursively interrupt its own handler. Normally, the kernel restores the previous mask when the handler returns (the `sigreturn` syscall does this). Using `longjmp` to escape the handler skips `sigreturn` entirely — the kernel never sees the handler "return", so the mask is left with **SIGINT permanently blocked**.

On the second iteration of `main`, line 30 `Kill(getpid(), SIGINT)` delivers SIGINT, but the kernel finds it blocked, sets the pending bit, and does *not* run the handler. `main` then continues past line 30 and prints

```
after SIGINT (never reached)
```

before returning normally with rc 0 — the restart loop is broken.

`sigsetjmp(env, 1)` saves the current signal mask into `env` (the second argument is "save mask?"). `siglongjmp` restores that saved mask atomically with the jump, so SIGINT is unblocked again before the next `Kill`.

---

## Problem 3: Scheduling Policy Comparison

### 3.1 Metrics table

| Policy | Avg. turnaround | Avg. response | CPU at t = 6 ms | CPU at t = 12 ms |
|---|---|---|---|---|
| FIFO | **8.0 ms** | **4.25 ms** | **A** | **D** |
| SJF | **7.75 ms** | **4.0 ms** | **A** | **D** |
| STCF | **5.5 ms** | **0.0 ms** | **D** | **A** |
| RR | **6.75 ms** | **0.75 ms** | **C** | **D** |

#### Trace details

**FIFO** runs jobs in arrival order, never preempts:

- A: 0 → 8, B: 8 → 10, C: 10 → 11, D: 11 → 15.
- Turnarounds: A=8, B=10−2=8, C=11−4=7, D=15−6=9 → avg 32/4 = 8.0.
- Responses: A=0, B=8−2=6, C=10−4=6, D=11−6=5 → avg 17/4 = 4.25.

**SJF (non-preemptive)** picks shortest *next* job at each completion:

- A: 0 → 8 (only one ready). At t=8, ready = {B(2), C(1), D(4)} → pick C.
- C: 8 → 9. At t=9, ready = {B(2), D(4)} → pick B.
- B: 9 → 11. D: 11 → 15.
- Turnarounds: A=8, B=11−2=9, C=9−4=5, D=15−6=9 → avg 31/4 = 7.75.
- Responses: A=0, B=9−2=7, C=8−4=4, D=11−6=5 → avg 16/4 = 4.0.

**STCF (preemptive SJF)** re-evaluates on every arrival:

- t=0: A starts (8 ms left).
- t=2: B arrives (2 < 6 left of A) → preempt. B: 2 → 4.
- t=4: C arrives (1 < 6 left of A) → C runs. C: 4 → 5.
- t=5: A only ready, A resumes (6 ms left).
- t=6: D arrives (4 < 5 left of A) → preempt. D: 6 → 10.
- t=10: A resumes (5 ms left). A: 10 → 15.
- Turnarounds: A=15, B=4−2=2, C=5−4=1, D=10−6=4 → avg 22/4 = 5.5.
- Responses: A=0, B=0, C=0, D=0 → avg 0.0.

**RR (2 ms quantum)**:

- t=0–2: A. At t=2, B arrives. Insert B at tail, then A at tail → queue = [B, A(6)].
- t=2–4: B (full quantum, finishes). At t=4, C arrives. Queue = [A(6), C].
- t=4–6: A. At t=6, D arrives. Insert D, then A → [C, D, A(4)].
- t=6–7: C (only 1 ms left, finishes early). Queue = [D, A(4)].
- t=7–9: D (full quantum). Queue = [A(4), D(2)].
- t=9–11: A. Queue = [D(2), A(2)].
- t=11–13: D (finishes). Queue = [A(2)].
- t=13–15: A (finishes).
- Turnarounds: A=15, B=4−2=2, C=7−4=3, D=13−6=7 → avg 27/4 = 6.75.
- Responses: A=0, B=0, C=6−4=2, D=7−6=1 → avg 3/4 = 0.75.

## 3.2 Best response time

**STCF** achieves the smallest average response time (0 ms). At every arrival, the new job is shorter than what is currently running (B=2 < A's 6 left, C=1 < A's 6 left, D=4 < A's 5 left), so STCF preempts immediately and the new job's first-run time equals its arrival time — response time is exactly 0 for every job.

STCF would do worse than RR on a workload of **multiple long jobs arriving close together**: e.g., three jobs each 100 ms arriving at t=0, t=1, t=2. STCF would let the first one run almost to completion before switching (no shorter alternative ever appears), giving the second and third huge response times. RR's 2 ms quantum would interleave them and give all three small response times.

## 3.3 Convoy effect on FIFO

Under FIFO, A (the only long job) sits at the head of the run for 8 ms. The three short jobs B, C, D arrive at t=2, 4, 6 but cannot run until t=8 even though they are individually short — B has to wait 6 ms to run for 2 ms, C waits 6 ms to run for 1 ms, D waits 5 ms to run for 4 ms. The single long job at the front "convoys" all the short jobs behind it, inflating the average response time to 4.25 ms (vs. 0 ms achievable by STCF on the exact same workload).

---

# Problem 4: MLFQ Trace

## 4.1 Schedule

| **Time (ms)** | **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** |
|---|---|---|---|---|---|---|---|---|---|
| CPU | A | **A** | **B** | **A** | **C** | **B** | **A** | **C** | **C** |

Step-by-step:

- **t=0–1.** A enters high queue. Runs 1 ms = full high slice → demoted to low (A has 3 ms left).
- **t=1–2.** High empty. A runs from low (2 ms slice). After 1 ms, B arrives at t=2 and preempts (high > low). A relinquished before its slice ended → stays at low; A goes to back of low queue with a fresh slice next time. (A has 2 ms left.)
- **t=2–3.** B on high (1 ms full). Demoted. Low = [A(2), B(1)].
- **t=3–4.** A on low (fresh 2 ms slice). After 1 ms, C arrives at t=4 and preempts. A → back of low (A has 1 ms left).
- **t=4–5.** C on high (1 ms full). Demoted. Low = [B(1), A(1), C(2)].
- **t=5–6.** B on low (2 ms slice, but only 1 ms of work). B finishes at t=6.
- **t=6–7.** A on low (1 ms of work, finishes early). A finishes at t=7.
- **t=7–9.** C on low (2 ms slice, full). C finishes at t=9.

Sanity check on counts: A appears 4 times (t=0, 1, 3, 6) ✓ runtime 4 ms; B appears 2 times (t=2, 5) ✓ runtime 2 ms; C appears 3 times (t=4, 7, 8) ✓ runtime 3 ms.

## 4.2 Metrics

| Job | Completion (ms) | Turnaround (ms) | Response (ms) |
|---|---|---|---|
| A | 7 | 7 − 0 = 7 | 0 |
| B | 6 | 6 − 2 = 4 | 0 |
| C | 9 | 9 − 4 = 5 | 0 |

- **Average turnaround:** (7 + 4 + 5) / 3 = **16 / 3 ≈ 5.33 ms**.
- **Average response:** (0 + 0 + 0) / 3 = **0 ms**.

## 4.3 Starvation without priority boost

**Workload.** Job A arrives at t = 0 and runs for 100 ms (CPU-bound). Job B_i arrives at t = i for i = 1, 2, 3, … and each B_i runs for exactly 1 ms.

**What happens.** A runs t=0–1 in the high queue, uses its full 1 ms slice, and is demoted to low. From t=1 onward, exactly one new B_i is in the high queue at every 1 ms boundary — each one runs the full 1 ms high slice, completes, and immediately the next B_(i+1) arrives. The high queue is *never* empty after t=1, so MLFQ Rule 1 forbids running A from the low queue. **A starves indefinitely.**

With Rule 5 (priority boost) every S ms, A would be promoted back to the top queue and get to run; without it, the policy has no way to recover.